# WEEK 06

Internship Report

MUHAMMAD MUTEE ULLAH
CODELOUNGE
Wapda town Lahore

# Contents


Introduction....2
Duration....2
Week 06....2
Overview....2
Project Structure:....3
Task Performed....3
1. Project Initialization & Setup....3
2. Audio/Video Engine Integration....3
3. Core Service Architecture....4
4. File & Permission Management....4
5. Initialization Pipeline....4
6. Database & Local Storage....5
7. Theme & UI Foundation....5
Learning Outcomes....5
Technical Skills Acquired:....5
1. Flutter Architecture Patterns....5
2. Media Playback Integration....5
3. Platform-Specific Development....6
4. Flutter Best Practices....6
5. Database & Storage....6
Key Understanding:....6
Challenges and Solutions....7
Conclusion....8

# Introduction

NYX Media Player is an advanced Flutter-based media player application designed for seamless audio and video playback. Week 06 focused on establishing the project foundation, setting up the core architecture, integrating media playback engines, and implementing essential backend services. This week laid the groundwork for a robust, production-ready media player with Android support.

# Duration

## Week 06

Initial development phase focusing on project setup, architecture design, and core service implementation

# Overview

NYX Media Player is built using the **MVVM (Model-View-ViewModel)** architectural pattern with Provider for state management. The project includes:

**Target Platforms**: Android, iOS, Windows, Linux, and Web

**Architecture**: Clean architecture with separation of concerns

**State Management**: Provider (v6.1.2)

**Media Engines**:

ExoPlayer on Android (via just_audio)

Native media_kit library for video playback

**Database**: SQLite (sqflite) for local media caching

**Key Dependencies**: 50+ packages including specialized media libraries

# Project Structure:

lib/

├── main.dart (Entry point with service initialization)

├── controllers/ (Business logic controllers)

├── core/ (Theme, navigation, utilities)

├── models/ (Data models: MediaItem, Playlist, MediaTag)

├── services/ (Core services: Audio player, Database, File handling)

├── viewmodels/ (UI state management)

├── views/ (UI screens)

└── widgets/ (Reusable UI components)

# Task Performed

## 1. Project Initialization & Setup

- Created Flutter project structure with standard directories
- Configured pubspec.yaml with 50+ dependencies
- Set up Android, iOS, Windows, Linux, and Web platforms
- Configured launcher icons and app metadata

## 2. Audio/Video Engine Integration

- Integrated **just_audio** (v0.9.43) for audio playback with ExoPlayer on Android
- Integrated **media_kit** family of packages for advanced video playback
- Configured platform-specific libraries:
  - Android: media_kit_libs_android_video

- iOS: media_kit_libs_ios_video
- Windows: media_kit_libs_windows_video
- Linux: media_kit_libs_linux

## 3. Core Service Architecture

- **AudioPlayerService**: Manages playback operations, queuing, and state
- **MediaDatabaseService**: SQLite integration for media caching
- **MediaTagService**: Handles user-edited media metadata
- **PlaybackManager**: Centralized playback coordination
- **PlaytimeService**: Tracks and manages playtime statistics
- **AudioServiceHandler**: Android foreground service integration

## 4. File & Permission Management

- Implemented **file_picker** for file selection
- Integrated **permission_handler** for runtime permissions
- Added **image_picker** for cover art selection
- Configured Android manifest for required permissions

## 5. Initialization Pipeline

- MediaKit initialization
- AudioService foreground service setup with timeout handling
- Service initialization sequence with proper dependency ordering
- System UI mode configuration (edge-to-edge UI)
- Persistent data loading (preferences, media tags, playlists)

### 6. Database & Local Storage

- SQLite setup with sqflite for media metadata caching
- path_provider integration for app document directories
- SharedPreferences for gesture preferences and user settings

### 7. Theme & UI Foundation

- Google Fonts integration for modern typography
- Theme controller setup for dynamic theming
- System UI overlay configuration

## Learning Outcomes

### Technical Skills Acquired:

#### 1. Flutter Architecture Patterns

- MVVM pattern implementation with Provider
- Service locator and dependency injection patterns
- Clean architecture separation of concerns

#### 2. Media Playback Integration

- Understanding of platform-specific media engines
- Android ExoPlayer capabilities and configuration
- Cross-platform media library compatibility
- Audio service foreground notification implementation

## 3. Platform-Specific Development

- Android service components and process management
- iOS notification handling
- Windows/Linux native library integration
- Permission handling across platforms

## 4. Flutter Best Practices

- Async initialization with proper timeout handling
- Service lifecycle management
- Android foreground service integration with timeout protection
- SystemUI and edge-to-edge UI configuration

## 5. Database & Storage

- SQLite database design for media metadata
- Local file system access and permissions
- Persistent preference storage

## Key Understanding:

- Media player applications require complex initialization sequences to prevent ANR (Application Not Responding) errors
- Cross-platform compatibility requires careful dependency management
- Proper service lifecycle management is critical for background playback
- Audio services need foreground notification support for Android 8+

# Challenges and Solutions

**Challenge 1: AudioService Hanging on Cold Start**

**Solution:**
Implemented an 8-second timeout with a fallback mechanism to prevent the white screen issue during app startup.

**Challenge 2: Platform-Specific Media Libraries**

**Solution:**
Utilized the media_kit abstraction layer along with platform-specific implementations to ensure smooth cross-platform media handling.

**Challenge 3: Android Foreground Service Requirements**

**Solution:**
Configured AudioServiceConfig with a proper notification channel and enabled an ongoing notification to comply with Android foreground service policies.

**Challenge 4: Permission Handling Across Platforms**

**Solution:**
Integrated permission_handler with platform-specific permission checks prior to performing file operations.

**Challenge 5: Dependency Initialization Order**

**Solution:**
Designed a structured initialization pipeline:
**MediaKit → AudioService → Services → UI**
to ensure proper dependency loading and avoid runtime issues.

**Challenge 6: Database Initialization on First Launch**

**Solution:**
Applied a lazy loading pattern for the database with automatic schema creation during the first app launch.

# Conclusion

Week 06 successfully established NYX Media Player's foundation with a robust architecture, integrated media playback engines, and implemented essential backend services. The project now has:

Professional audio/video playback capabilities

Proper service lifecycle management with timeout handling

Database and persistent storage infrastructure

Clean MVVM architecture with Provider state management

Permission handling and file system access

Modern UI foundation with Google Fonts integration

The groundwork is solid for building the user-facing features in Week 07, including the media library UI, player controls, playlists, and advanced media management features. The initialization pipeline is optimized to prevent ANR errors and handle edge cases gracefully.

**End of Week 06 Report**